

Performance Testing

Date	30 June 2026
Team ID	
Project Name	Credit Card Approval Prediction System
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Manual Testing (Flask Local Server)
Type of Testing	Functional Performance Testing
Target Module / API	Credit Card Eligibility Prediction (/result)
Test Environment	Local Flask Server (Windows, Python 3.12)
Test Date	10-07-2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Valid applicant prediction	1	30 sec	Prediction displayed successfully

2	Invalid applicant prediction	1	30 sec	Rejected prediction displayed
3	Multiple consecutive predictions	1	60 sec	Application responds correctly
4	Form validation	1	30 sec	Invalid inputs handled properly



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.82 sec	Pass	Fast response during local testing
2	Response Time (Max)	< 5 seconds	1.48 sec	Pass	Stable response under repeated requests
3	Throughput (Req/sec)	-	2 req/sec	Pass	Suitable for singleuser Flask application
4	Error Rate	< 1%	0%	Pass	No request failures observed
5	CPU Utilization	< 80%	22%	Pass	Low CPU usage during testing
6	Memory Utilization	< 80%	31%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings:

The Credit Card Approval Prediction System responded quickly and consistently during local testing. Predictions were generated successfully with low response time and no application crashes.

Bottlenecks Identified:

No significant performance bottlenecks were observed during local testing. The application handled repeated prediction requests without noticeable delay.

Optimization Steps Taken:

Performance was improved using a trained Random Forest model, optimized preprocessing with StandardScaler, and efficient model loading using Joblib.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

